Entrega de Desenvolvimento de Aplicações Dinâmicas

Descreve cada requisito da disciplina que o projeto atende e leva, por link direto, ao arquivo ou à pasta que o comprova na branch main.

Período	Responsável
2026	**Equipe do projeto**

Sumário

O que há neste documento

01
Visão geral 3
O que a disciplina pede e como ler o documento

02
Requisitos mínimos, base do código 5
Vite, organização, componentes e tipos

03
Requisitos mínimos, dados e navegação 7
Serviços, estado, efeitos, listas e rotas

04
Requisitos mínimos, interface e histórico 9
Acessibilidade, feedback ao usuário e commits

05
Requisitos extras 10
Context API, hook com cancelamento, lazy e proteção contra XSS

06
Repositórios da disciplina 12
Os quatro repositórios da entrega web

01

A entrega web atende os requisitos descritos com evidência no GitHub

O que a disciplina pede e como este documento se organiza?

O que a disciplina pede

A disciplina de Desenvolvimento de Aplicações Dinâmicas pede uma aplicação web em React com TypeScript, organizada em componentes, serviços, rotas e recursos de acessibilidade. Há requisitos mínimos, que valem a nota 7,0, e requisitos extras.

O que o projeto entregou

A interface web do projeto é o repositório web-app, construído com Vite, React e TypeScript. A entrega web reúne quatro repositórios, o web-app, a biblioteca de componentes lib-web, o Cloudflare Worker worker-failover e a interface de administração do QA web-sandbox. O código do web-app é organizado por funcionalidade, com serviços de API por domínio, tipos por domínio, estados de carregamento e erro, e um histórico de commits no padrão Conventional Commits. Este documento descreve os requisitos tratados e mostra, para cada um, em qual repositório da entrega ele aparece.

Como ler este documento

Cada requisito tem um identificador (M para mínimo e E para extra), o enunciado da disciplina, a descrição do que o projeto entrega e um ou dois links diretos. Todos os links apontam para a branch main e abrem o arquivo ou a pasta que comprova o requisito. Quando outro repositório da entrega comprova o requisito por conta própria, o texto cita esse repositório pelo nome. Os requisitos aparecem em grupos por tema, na ordem em que o código é lido.

Requisitos descritos neste documento

Identificador	Requisito	Tipo
M1	Vite com React e TypeScript	Mínimo
M2	Organização de componentes e páginas	Mínimo
M3	Componentes de responsabilidade única	Mínimo
M4	Tipos TypeScript por domínio	Mínimo
M5	Comunicação com a API em serviços	Mínimo

Identificador	Requisito	Tipo
M6	Estado local com useState	Mínimo
M7	useEffect com dependências	Mínimo
M8	Listas com chave estável	Mínimo
M10	Rotas com React Router e página de erro	Mínimo
M11	Acessibilidade	Mínimo
M12	Feedback de operações assíncronas	Mínimo
M14	Commits em Conventional Commits	Mínimo
E1	Context API	Extra
E3	Hook customizado com AbortController	Extra
E4	React.lazy e Suspense	Extra
E7	Proteção contra XSS	Extra

02

A base do web-app usa Vite, React e TypeScript, organizada por funcionalidade

Como o código está montado, dividido e tipado?

M1 Vite com React e TypeScript

Projeto criado com Vite e template React TypeScript; todos os arquivos de `src/` em `.ts` ou `.tsx`, proibido `.js` e `.jsx`.

O web-app foi criado com Vite, React e TypeScript, como mostra o package.json. A pasta `src/` não tem arquivo `.js` nem `.jsx`.

O lib-web e o web-sandbox também usam Vite, React e TypeScript, sem arquivos `.js` ou `.jsx` em `src`. O worker-failover tem o código-fonte em TypeScript, sem arquivos `.js` em `src`, e cinco arquivos de teste `*.test.ts`.

- package.json em web-app
- index.ts em worker-failover

M2 Organização de componentes e páginas

Componentes em `src/components/` e páginas em `src/pages/`, cada um em pasta própria com `index.tsx`.

O código é organizado por funcionalidade. Cada domínio tem uma pasta em `src/features/` com suas páginas (`pages/`) e seus componentes (`components/`), e os componentes reutilizáveis ficam em `src/lib/components/`.

- pasta src/features em web-app
- pasta src/lib/components em web-app

M3 Componentes de responsabilidade única

Mínimo de 6 componentes com responsabilidade única.

O web-app tem dezenas de componentes reutilizáveis, cada um com uma função só, como a barra lateral (Sidebar), o gráfico (Chart), a seleção (Select) e o menu de contexto (ContextMenu).

O lib-web reúne os componentes compartilhados em pastas por categoria (`ui`, `feedback` e `layout`), como Skeleton, Combobox e OperationalTable.

- Chart.tsx em web-app
- pasta src/components em lib-web

M4 Tipos TypeScript por domínio

Interfaces TypeScript para props e entidades do domínio em `src/types/`, por arquivo de domínio; proibido `any`.

As interfaces e os tipos das entidades ficam separados por domínio. Os tipos compartilhados estão em `src/lib/shared/types`, e cada funcionalidade tem o seu arquivo `.d.ts`, como o analytics.d.ts. A busca por `any` fora dos testes não encontra nenhuma ocorrência.

No lib-web, os tipos das props ficam em arquivos `.types.ts` ao lado de cada componente, como o Skeleton.types.ts.

- analytics.d.ts em web-app
- Skeleton.types.ts em lib-web

03

Serviços, estado, efeitos, listas e rotas seguem padrões definidos

Como o web-app busca dados, guarda estado e navega entre páginas?

M5 Comunicação com a API em serviços

Toda comunicação com API isolada em `src/services/`, por recurso, com retorno `Promise<Tipo>` e `try/catch`; chaves e URLs em variáveis de ambiente.

A comunicação com a API fica em serviços por domínio, no padrão `src/features/<domínio>/<domínio>.service.ts`, com retorno tipado. O analytics.service.ts é um exemplo, e o company.postal.service.ts tem teste próprio. As URLs vêm de variáveis `VITE_` listadas no .env.example, e o arquivo `.env` fica fora do versionamento pelo .gitignore.

No web-sandbox, o keepalive.service.ts tem a função `wake` com retorno `Promise<WakeResult>` e bloco `try/catch`.

- .env.example em web-app
- keepalive.service.ts em web-sandbox

M6 Estado local com useState

Estado local com `useState`, campos relacionados agrupados em um objeto; proibida mutação direta de arrays e objetos.

O formulário de cadastro de empresa, em CompanyOnboarding.tsx, usa `useState` e agrupa os campos de endereço em um único objeto de estado.

- CompanyOnboarding.tsx em web-app

M7 useEffect com dependências

`useEffect` com array de dependências explícito e correto.

O ESLint do projeto ativa a configuração recomendada do plugin eslint-plugin-react-hooks, que aplica as regras dos hooks a todo o código. A página AnalyticsPage.tsx usa `useEffect` com o array de dependências declarado.

- eslint.config.js em web-app
- AnalyticsPage.tsx em web-app

M8 Listas com chave estável

Listas dinâmicas com `key` de identificador único; proibido `key={index}`.

O chat monta a chave de cada mensagem com o usuário e a hora, o gráfico usa o nome da série e o rótulo da linha, e as opções do Select usam o valor da opção.

- ChatLayout.tsx em web-app
- Select.tsx em web-app

M10 Rotas com React Router e página de erro

Mínimo de 3 rotas com React Router, navegação por `<Link>` ou `useNavigate`, e rota curinga com página de erro e botão de retorno.

O AppRoutes.tsx define as rotas do aplicativo com React Router e inclui uma rota curinga (`path="*"`) que mostra a página NotFound. A página tem um botão de retorno que leva à página inicial com `useNavigate`.

No web-sandbox, o App.tsx declara nove rotas com React Router, entre elas uma rota com parâmetro de sessão e uma rota curinga que redireciona ao catálogo.

- NotFound.tsx em web-app
- App.tsx em web-sandbox

04

Acessibilidade, feedback ao usuário e commits padronizados

Que cuidados de uso e de processo o projeto mostra?

M11 Acessibilidade

Acessibilidade mínima WCAG 2.1 AA, com botões, `label` e `htmlFor`, `alt`, contraste, `role="alert"`, teclado e foco visível.

No web-app, a pasta `src/lib/shared/accessibility` traz a geração de texto alternativo por IA para imagens, `role="alert"` aparece em 36 arquivos e o foco visível (`focus-visible`) em 28 arquivos.

No lib-web, o Access associa cada campo ao rótulo com `label` e `htmlFor`, o FieldFeedback usa `role="alert"` e o LogoLoader, `role="status"`.

- Access.tsx em lib-web
- pasta src/lib/shared/accessibility em web-app

M12 Feedback de operações assíncronas

Feedback visual para operações assíncronas com três estados (carregando, sucesso, erro); proibido `alert()` e `console.log` como único tratamento.

No web-app, a página AnalyticsPage.tsx mostra o carregamento, uma mensagem de erro com `role="alert"` se a busca falha e os dados quando ela termina.

No web-sandbox, o KeepalivePage.tsx usa `role="status"` e o AiFlowPage.tsx usa `role="alert"`.

- AnalyticsPage.tsx em web-app
- KeepalivePage.tsx em web-sandbox

M14 Commits em Conventional Commits

Mínimo de 20 commits em Conventional Commits, cobrindo ao menos 4 semanas.

O histórico do web-app tem mais de 370 commits entre 02/06/2026 e 02/11/2026, mais de 200 com prefixo do Conventional Commits, e o workflow pr-title-lint.yml recusa título de PR fora do padrão.

- histórico de commits do web-app
- pr-title-lint.yml em web-app

05

Os extras entregues cobrem contexto, hook com cancelamento, carregamento sob demanda e proteção contra XSS

Que recursos além do mínimo a entrega web traz?

E1 Context API

Context API com Provider dedicado, hook de consumo e tipagem; demonstrar o problema de prop drilling.

O contexto da empresa ativa, em ActiveContext.tsx, tem um Provider dedicado (`ActiveContextProvider`), uma interface tipada para o valor do contexto e um hook de consumo (`useActiveContext`).

- ActiveContext.tsx em web-app

E3 Hook customizado com AbortController

Hook customizado com retorno tipado, loading, erro e cancelamento via `AbortController`.

O hook `useMarketplaceCatalog`, em marketplace.hooks.ts, é genérico e devolve os itens tipados, o estado de erro e uma função para tentar de novo. A busca é cancelada com `AbortController` quando o componente sai da tela.

No web-sandbox, o SimulatorPage.tsx cancela a execução em andamento com `AbortController`.

- marketplace.hooks.ts em web-app
- SimulatorPage.tsx em web-sandbox

E4 React.lazy e Suspense

`React.lazy` e `Suspense` por página, demonstrado no Network.

No web-sandbox, o App.tsx carrega as páginas com `React.lazy` e as envolve em `Suspense`, com um indicador de carregamento. No web-app, o DocumentViewer.tsx usa os dois nos visualizadores de PDF e de DOCX.

- App.tsx em web-sandbox
- DocumentViewer.tsx em web-app

E7 Proteção contra XSS

Proteção contra XSS, com `dangerouslySetInnerHTML` proibido para dados externos.

O código de `src/` não usa `dangerouslySetInnerHTML`, então nenhum dado externo é inserido como HTML. O arquivo sanitization.utils.ts reúne a função de sanitização disponível para esse caso.

- sanitization.utils.ts em web-app

Entregas adicionais

A entrega web inclui uma biblioteca de componentes publicada como pacote, com tokens de design e Storybook, um Worker na borda que escolhe o idioma da visita e mostra uma página temporária quando a origem está indisponível e uma interface de administração do QA com proxy autenticado do Swagger. No web-app, o Storybook tem mais de oitenta arquivos de story e a internacionalização cobre português, inglês e espanhol.

Link de exemplo, lib-web na branch main.

06

A entrega web reúne quatro repositórios, e cada um aparece nos requisitos que comprova

Onde fica o código descrito neste documento?

Repositório	Papel na disciplina	Raiz
web-app	Interface web do projeto, em Vite, React e TypeScript. Reúne o código, a documentação de arquitetura e os workflows citados neste documento.	web-app na branch main
lib-web	Biblioteca de componentes React com tokens de design, internacionalização e Storybook, publicada como pacote.	lib-web na branch main
worker-failover	Cloudflare Worker em TypeScript que escolhe o idioma, encaminha a navegação para a origem e mostra uma página temporária quando a origem está indisponível.	worker-failover na branch main
web-sandbox	Interface React de administração do QA, com servidor Fastify e proxy autenticado do Swagger.	web-sandbox na branch main



Entrega acadêmica.